

CMSC411: Computer Systems Architecture

Project 2 : Tomasulo's Algorithm

Spring 2024

Project Overview

In this project, we will be designing a simulator that implements a simplified version of Tomasulo's algorithm. You are asked to design the register alias table (RAT) and reservation stations (REST).

Each line of the trace file (e.g., MEM 1 2 3) is in the following format: **OP RD RS RT**.

- **OP** indicates the operation type. There will be different types of instructions, each with a different latency (e.g., ADD takes 2 cycles, DIV takes 15 cycles, MEM takes 20).
- **RD** is the destination register number (e.g., 1 for R1)
- **RS** is the first source register number. If there is none (for example, an immediate value), the “number” is -1.
- **RT** is the second source register number.

```
MEM 1 2 3
MEM 4 5 6
MEM 7 8 9
```

Simulator Overview

- schedulersim.cpp: You need to **make your changes in this file**. Follow these twelve numbers in the code to see the parts you need to complete: 1, 2.1, 2.2, 3.1, 3.2, 4.1, 4.2, 4.3, 4.4, 5.1, 5.2, 5.3.
- schedulersim_driver.cpp: This file includes the main() function and runs the simulator for you. You **don't need to change** anything in this file. But feel free to read it to better understand the code.
- schedulersim.hpp: This file includes the definition of the functions, structs, etc. You **don't need to change** anything in this file. But feel free to read it to better understand the code.
- Makefile: This is the makefile that you need to run to compile your code:
 - \$make
 - \$make clean
- /test-traces and /test-output: These two folder respectively include a group of test traces that your simulator will read and the output that it must generate if your simulator works correctly.
- /traces: This folder includes real-world benchmarks that you will use to optimize your code for.
- validate-test-traces.sh: You will be running this script to test your code. Basically, this script runs schedulersim for all the traces in /test-traces, writes the outputs in /myoutput, compares everything in /test-output with /myoutput and prints out the differences.

What to do

- We have provided you with an implementation of REST and RAT.
- Some functions (e.g., `add_entry`, `init_table`, `is_empty`, `fire_ready`, `count_active`, `complete_insts`, `complete_insts`, `free_table`) in REST, however, are not completed.
- You may either choose to complete these functions or redefine your brand new REST and RAT in a way you want.

1.1. scheduler_per_fu_init

- This function initializes the scheduler.
- Similar to the code provided for scheduler_unified_init, you need to set the num_regs, initialize the rat, set the rs, and initialize the reservation stations.
- To initialize the reservation stations, you may call the init_table from REST class that takes the size of the table and the start index.

2.1. add_entry for REST

- Here you are adding an entry to the REST table.
- Therefore, as you can see, the provided code iterates over the table to find an empty entry first:

```
for (int i = 1; i <= size; i++) {  
    if (!is_valid(i)) {
```
- Your job is
 - to set that free entry of the REST to the instruction (op, dest, scr1, and scr2)
 - also to set the rat with the index of the REST that you're putting your instruction in
 - To do this you may use the set_reg of RAT class

2.2. scheduler_try_issue

- Now that you completed the `add_entry` for the reservation stations in section 2.1, you can call that function to actually add the entry corresponding to the argument of `scheduler_try_issue` to your specific reservation station
- You may use the provided code for the unified REST as an example
- You may use a switch-case to decide the REST based on op

3.1. complete is_empty for REST

- This function simply checks to see if there is **any** valid entry in the table.
- In other words, if there is any valid entry, you need to return false.
- You may call is_valid function for each entry.

3.2. scheduler_completed

- This function checks if all instructions have been drained from the pipeline
- Now that you have completed the is_empty in 3.1, you can just call it to see if all the reservation stations are empty

4.1. fire_ready for REST

- Here, you are going to start executing any valid instruction in the table with ready sources
- To do so, we have a for loop that iterates over the table
- For each entry you need to check if
 - It is valid, and
 - Both of the operands are **ready**, and
 - The start cycle is zero (meaning that it's **not already started**)
- If you found such an entry, you need to then set the corresponding flag of the functional unit of that type to true
 - To do so, you may use a switch-case based on the op type of that entry

4.2., 4.3., and 4.4. complete scheduler_start_ready

- 4.2: update the num_fired based on the number of fired (started execution) instructions (you may call fire_ready)
- 4.3: update p_stats->max_fired
- 4.4: update pstats->max_active for each type of operation

5.1. complete_insts for REST

- For every instruction in the table check if its execution is done, if yes, then write the results back
- The for loop for iterating over the entries of the table is provided
- For each entry, you need to check **if the entry is valid and if its execution has started**
 - Then, check if it is finished, to do which, **you can get the expected latency by calling get_inst_latency**
 - If you found them, then free the RAT (you may call set_reg)
 - Invalidate the REST entry
 - And write back the results (you may call broadcast_complete)

5.2. and 5.3 scheduler_clear_completed

- 5.2: update the num_completed based on the number of completed instructions (you may call complete_insts similar to the template code for unified reservation station)
- 5.3: update p_stats->max_completed

Statistics

The simulator outputs the following statistics after completion of the run:

1. Number of instructions
2. Number of cycles
3. Instructions Per Cycle (IPC)
4. Number of times the issue was stalled (issue failed)
5. Maximum number of instructions started at once
6. Maximum number of instructions completed (wrote back) at once
7. Maximum number of instructions active per FU

Experiments

For each benchmark in the traces directory, vary any input parameters (e.g., RS type, entries per RS) to design an instruction scheduler subject to the following goals:

1. You have a total budget of 10 entries for the reservation stations (10 all in the unified RS, or X for each RS that sums up to 10 entries)
2. The scheduler should have the highest IPC in the end. You may vary any parameter

Important notes:

- DO NOT add screenshots of your terminal to report the outputs.
- Similarly, DO NOT copy+paste the output of your program into your report.
- DO NOT just report one number saying that it's the best. Show how a performance metric has changes with a parameter
- Add your outputs in tables or show them in diagrams

Grading Rubric

- 0% You do not hand in anything
- Up to +36% Your simulator doesn't run, does not work, but you hand in significant commented code (depending on how many parts of the code you completed you gain maximum 36%, 3% per each)
- +48% Your simulator matches the validation outputs (8% per output)
- +16% You ran all experiments and found the best-performing schedulers
- *Important note:* Late submissions will be deducted 25% per day